

Sequence-to-Sequence Models and Attention

Scaler School of Technology | Deep Learning I | Week 6 — Session 2/2 | Post-Lecture Review & Practice



Key Takeaways

1 Encoder-Decoder Compresses Everything Into One Vector

The basic seq2seq architecture uses a single fixed-size context vector to bridge encoder and decoder. This works for short sequences but fails for long ones — the “information bottleneck.” Early words in long inputs get progressively forgotten.

2 Attention Lets the Decoder Look Back at Every Position

Instead of one context vector, attention computes a **fresh weighted sum** of all encoder states at each decoding step. The weights (alignment scores) are learned end-to-end. Bahdanau uses an additive neural network for scoring; Luong uses a simple dot product.

3 Query-Key-Value Is the Universal Attention Framework

Modern attention: a Query (what you're looking for) is compared against Keys (what's available) via dot product, then used to retrieve a weighted combination of Values. This “soft lookup” is differentiable and trainable. It's **exactly** what Transformers use.

Test Your Understanding

Try answering without looking at slides. Answers on page 3.

- Q1.** In a basic encoder-decoder, why does the decoder struggle with long input sentences? Be specific about the architectural reason.
- Q2.** Write the three steps of the attention mechanism at decoder step t . What are the inputs and output of each step?
- Q3.** Why is “soft” attention trainable with backpropagation while “hard” attention is not?
- Q4.** What is the key difference between Bahdanau (additive) and Luong (dot-product) attention? When would you prefer each?
- Q5.** Explain how attention can be viewed as a Query-Key-Value lookup. What plays the role of Q, K, and V in seq2seq translation?

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned.

Exercise 1: Seq2Seq for Reversed Strings

Build an encoder-decoder that reverses strings (e.g., “hello” → “olleh”). Train on random strings of length 5–10. Then test on strings of length 15–20. Does accuracy drop? Add attention and compare. Plot accuracy vs. input length for both models.

Exercise 2: Compare Attention Variants

Implement three attention scoring functions: (a) Bahdanau additive, (b) Luong dot-product, (c) Luong general (bilinear). Train all three on the date conversion task from the lecture notebook. Compare: training curves, final accuracy, and training time per epoch. Which converges fastest? Which achieves best accuracy?

Exercise 3: Attention Heatmap Analysis

Using the trained attention model from the notebook, generate attention heatmaps for 10 test examples. For each, annotate whether the alignment is “monotonic” (diagonal) or “non-monotonic” (reordered). What input formats produce non-monotonic attention? Why?

Challenge Problem

Build a Tiny Machine Translation System

Build a character-level seq2seq model with attention for English→Pig Latin translation (e.g., “hello” → “ellohay”, “string” → “ingstray”):

1. Generate a dataset of 10,000 English words and their Pig Latin equivalents programmatically
2. Build encoder-decoder with Bahdanau attention
3. Train and evaluate — report character-level accuracy and full-word accuracy
4. Visualize attention heatmaps for 5 words — does the model learn the reordering rule?

Expected output: Accuracy table + 5 attention heatmaps showing learned alignment patterns.

Bonus: Add a bidirectional encoder. Does it improve accuracy? Why might bidirectionality help for this task?

- **Seq2Seq** — Sutskever et al., 2014, “Sequence to Sequence Learning with Neural Networks.” The original encoder-decoder paper. Read Section 2.
- **Bahdanau Attention** — Bahdanau et al., 2015, “Neural Machine Translation by Jointly Learning to Align and Translate.” The paper that introduced attention. Section 3 is key.
- **Luong Attention** — Luong et al., 2015, “Effective Approaches to Attention-based NMT.” Compares dot-product, general, and concat scoring functions.
- **Preview** — Vaswani et al., 2017, “Attention Is All You Need.” Next week’s topic. Skim Section 3.2 (Scaled Dot-Product Attention) for a preview.

Online Resources & Tools

- **Jay Alammar’s Blog** — “Visualizing A Neural Machine Translation Model” — best visual explanation of attention
- **PyTorch Seq2Seq Tutorial** — Official tutorial building translation with attention step-by-step
- **The Annotated Transformer** — Harvard NLP group’s line-by-line implementation with explanations
- **Tensor2Tensor** — Google’s library for seq2seq experiments (now legacy but educational)
- **OpenNMT** — Open-source neural MT with attention implementations
- **Lilian Weng’s Blog** — “Attention? Attention!” — comprehensive survey of attention variants

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about softmax?** It converts raw scores into a probability distribution (sums to 1). Formula: $\text{softmax}(x_i) = e^{x_i} / \sum_j e^{x_j}$. High scores get high probabilities.
- **Fuzzy on hidden states in RNNs?** Revisit Session 10. The hidden state h_t is the network’s “memory” at time t — it encodes everything seen up to that point.
- **Not sure what “end-to-end training” means here?** The entire encoder + decoder + attention is trained together with one loss (cross-entropy on output tokens). Alignment is never supervised directly.
- **Confused about teacher forcing?** During training, we feed the correct previous token to the decoder (not its own prediction). This stabilizes early training. At test time, we use the model’s own predictions.

Answers — Test Your Understanding

- A1.** The decoder only receives the encoder’s final hidden state — one fixed-size vector. This vector has limited capacity and early input words are “far away” in the computation graph, so their information fades.
- A2.** (1) Compute scores $e_{ti} = \text{score}(s_t, h_i)$ for all encoder states. (2) Normalize: $\alpha_{ti} = \text{softmax}(e_t)$. (3) Weighted sum: $c_t = \sum_i \alpha_{ti} h_i$. Input: decoder state + encoder states. Output: context vector c_t .
- A3.** Soft attention uses a continuous weighted sum (differentiable). Hard attention uses argmax to select one position (not differentiable — requires REINFORCE or similar RL methods to train).
- A4.** Bahdanau: $v^\top \tanh(W_{ss} + W_h h)$ — learnable, more expressive, slower. Luong dot: $s^\top h$ — no params, faster, scales better to large dims. Prefer additive for small models, dot-product for large.
- A5.** Query = current decoder state s_t (what we’re looking for). Keys = encoder hidden states h_i (what each position advertises). Values = also encoder states h_i (information retrieved). Attention weights = similarity(Q, K).